13

WHITE PAPER · WORK, ADOPTION & JUDGMENT

The Exception Path Is the Real Automated Workflow

Design recovery before declaring a workflow automated

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

A workflow is governed when its exception path has an owner, a boundary, an evidence trail, and a recovery decision.

The happy path demonstrates throughput. The exception path demonstrates whether the operation can detect what falls outside the rules, route it to an accountable owner, bound the consequence, preserve evidence, recover, and learn. This paper gives workflow owners, automation leaders, AI program owners, operations executives, and risk teams a practical the six-part exception path for making the next commitment explicit.

01 Classify exceptions: An exception taxonomy should describe why ordinary processing is unsafe or impossible and what kind of response is required.

02 Detect and contain: The system must recognize boundary conditions early and move into a safe state.

03 Route with context: A human handoff needs the case, reason, evidence, attempted actions, affected systems, and deadline.

04 Assign response ownership: Each class needs an owner for the case and an owner for the recurring condition behind it.

CONTENTS

The argument

The happy path demonstrates throughput. The exception path demonstrates whether the operation can detect what falls outside the rules, route it to an accountable owner, bound the consequence, preserve evidence, recover, and learn.

Automation proposals naturally emphasize the common path: complete input, known category, available dependency, permitted action, successful response. Operations are defined by what happens outside that path.

Missing data, conflicting records, ambiguous classification, changed policy, unavailable systems, unusual customer circumstances, and unsafe outputs are not peripheral. They are the cases that determine trust and support cost.

NIST's risk-management framework and Microsoft's red-team experience support a disciplined approach to testing, monitoring, response, and learning [1][2]. The exception-path model makes those ideas concrete inside a workflow.

This paper is written for workflow owners, automation leaders, AI program owners, operations executives, and risk teams. It offers the six-part exception path as a management instrument: a way to connect operating evidence to the next consequential choice. The cited sources provide external context; the framework and its application are Marco Policani's synthesis. It is not a predictive score and does not replace professional, legal, security, financial, or domain judgment.

Why the conventional approach breaks

Viewed from the portfolio, this is a resource-allocation problem before it becomes a delivery problem. Conventional governance tends to separate planning, approval, delivery reporting, and operating ownership. That separation allows a premise to pass through several forums without any one forum owning the decision it implies. The project or workflow then carries an assumption that was acknowledged socially but never accepted operationally.

The organization may call this execution risk, yet the failure began when authority and evidence were separated at the decision point. The answer is not heavier control at every step. It is to place a clear, proportionate test at the point where the organization increases money, capacity, access, dependency, or action authority.

The six-part exception path

The model has 6 connected elements. Each makes a different condition visible, but they should be reviewed together because strength in one area does not cancel a missing obligation in another. The model is designed for a short executive conversation supported by inspectable evidence, not a long maturity assessment.

EXHIBIT 1

The six-part exception path connects evidence to action

1. **Classify exceptions**
   An exception taxonomy should describe why ordinary processing is unsafe or impossible and what kind of response is required.
2. **Detect and contain**
   The system must recognize boundary conditions early and move into a safe state.
3. **Route with context**
   A human handoff needs the case, reason, evidence, attempted actions, affected systems, and deadline.
4. **Assign response ownership**
   Each class needs an owner for the case and an owner for the recurring condition behind it.

5 **Recover to a known state**

Resolution must restore data, systems, customer commitments, and workflow state—not simply close a ticket.

6 **Learn and redesign**

Exception evidence should change rules, data, interfaces, training, permissions, and the definition of eligible work.

Classify exceptions

An exception taxonomy should describe why ordinary processing is unsafe or impossible and what kind of response is required. A strong operating model makes the hidden negotiation explicit while options are still available.

Every failure enters one generic manual-review queue, hiding severity, recurrence, and the skills needed to resolve it. A polished status can coexist with this failure because status describes activity; it does not prove the premise that authorizes reliance.

Create classes for missing or conflicting data, low confidence, policy conflict, dependency failure, unauthorized request, suspected harm, and novel case. The aim is not certainty. It is a justified next commitment with an observable basis and a viable way back.

Frequency, consequence, detection method, response skill, recurrence, and whether the class was anticipated. Measures should expose changes in the operation rather than reward production of artifacts. Activity can support the story; it cannot substitute for the result.

Separate routine recoverable cases from events requiring immediate containment or policy decision. When the premise is weak, narrowing is often a better decision than forcing a binary choice between full approval and cancellation.

A tabletop review of classify exceptions should use the observed break as its scenario: Every failure enters one generic manual-review queue, hiding severity, recurrence, and the skills needed to resolve it. Participants then apply the proposed response. They create classes for missing or conflicting data, low confidence, policy conflict, dependency failure, unauthorized request, suspected harm, and novel case. The review identifies where authority, information, time, or recovery would run out. It is useful only if it changes the boundary or confirms who will act when the condition appears.

The minimum record for classify exceptions is anchored in actual proof. It includes frequency, consequence, detection method, response skill, recurrence, and whether the class was anticipated. The record also carries the choice supported by that proof: Separate routine recoverable cases from events requiring immediate containment or policy decision. Keeping evidence and decision together lets a later owner distinguish a deliberate residual risk from a premise that nobody examined. It also gives the team a clear reason to reopen the choice when the underlying facts change.

Across the work governed by workflow owners, automation leaders, AI program owners, operations executives, and risk teams, occurrences of this pattern should be compared rather than treated as unrelated project stories. The positive standard is: An exception taxonomy should describe why ordinary processing is unsafe or impossible

and what kind of response is required. If the same failure recurs, leaders can change delegation, capacity, intake, policy, or the intervention itself. That portfolio response is usually more valuable than asking each team to absorb another local workaround.

Two questions test classify exceptions. What evidence would show that the condition is no longer adequate? Would that evidence arrive early enough to preserve a feasible choice? The intended choice is clear: Separate routine recoverable cases from events requiring immediate containment or policy decision. A weak answer to either question calls for a smaller commitment, an earlier signal, or a safer fallback. A strong answer earns movement without claiming that uncertainty has disappeared.

Detect and contain

The system must recognize boundary conditions early and move into a safe state. Teams move faster when uncertainty is bounded honestly than when it is concealed inside an optimistic plan.

It continues with a plausible guess, partially completes an action, or fails silently because confidence and policy checks are not operationalized. Under pressure, the workaround is predictable: capable people compensate, local decisions proliferate, and the formal record stops describing how the operation actually runs.

Set observable rules, uncertainty thresholds, transaction limits, dependency checks, and containment behavior for each class. The design should state what remains unknown. Acknowledged uncertainty can be bounded; disguised uncertainty is inherited by the operation.

Detection rate, false positives and negatives, time to containment, partial actions, and unobserved failures found later. Where consequence is high, independent checking or cross-functional challenge reduces the risk that advocacy becomes its own proof.

Stop the case—and sometimes the workflow—when detection or containment is unreliable. Escalation is appropriate when authority is missing, not as a routine substitute for clear delegation.

A tabletop review of detect and contain should use the observed break as its scenario: It continues with a plausible guess, partially completes an action, or fails silently because confidence and policy checks are not operationalized. Participants then apply the proposed response. They set observable rules, uncertainty thresholds, transaction limits, dependency checks, and containment behavior for each class. The review identifies where authority, information, time, or recovery would run out. It is useful only if it changes the boundary or confirms who will act when the condition appears.

The minimum record for detect and contain is anchored in actual proof. It includes detection rate, false positives and negatives, time to containment, partial actions, and unobserved failures found later. The record also carries the choice supported by that proof: Stop the case—and sometimes the workflow—when detection or containment is unreliable. Keeping evidence and decision together lets a later owner distinguish a deliberate residual risk from a premise that nobody examined. It also gives the team a clear reason to reopen the choice when the underlying facts change.

Across the work governed by workflow owners, automation leaders, AI program owners, operations executives, and risk teams, occurrences of this pattern should be compared rather than treated as unrelated project stories. The positive standard is: The system must recognize boundary conditions early and move into a safe state. If the

same failure recurs, leaders can change delegation, capacity, intake, policy, or the intervention itself. That portfolio response is usually more valuable than asking each team to absorb another local workaround.

Two questions test detect and contain. What evidence would show that the condition is no longer adequate? Would that evidence arrive early enough to preserve a feasible choice? The intended choice is clear: Stop the case—and sometimes the workflow—when detection or containment is unreliable. A weak answer to either question calls for a smaller commitment, an earlier signal, or a safer fallback. A strong answer earns movement without claiming that uncertainty has disappeared.

Route with context

A human handoff needs the case, reason, evidence, attempted actions, affected systems, and deadline. The practical failure occurs between the formal approval and the ordinary pressure of running the work.

Reviewers receive a vague alert and reconstruct the history across logs, messages, and systems before they can act. The happy path can survive this weakness. Ordinary variation exposes it: a disputed input, absent owner, competing commitment, or case that falls outside the assumed rule.

Package the decision context and send it to the role with authority and competence, with a fallback when that role is unavailable. The smallest credible control is the one that changes behavior when the evidence is weak. Anything else is commentary.

Complete handoff rate, time to acceptance, clarification loops, misroutes, queue age, and escalation use. Qualitative observation belongs beside quantitative measures when workflow behavior, adoption, or judgment explains why the number moved.

Redesign routing when review effort is dominated by reconstructing context. A stop or rollback is evidence that the control works when it protects greater value elsewhere in the portfolio.

A tabletop review of route with context should use the observed break as its scenario: Reviewers receive a vague alert and reconstruct the history across logs, messages, and systems before they can act. Participants then apply the proposed response. They package the decision context and send it to the role with authority and competence, with a fallback when that role is unavailable. The review identifies where authority, information, time, or recovery would run out. It is useful only if it changes the boundary or confirms who will act when the condition appears.

The minimum record for route with context is anchored in actual proof. It includes complete handoff rate, time to acceptance, clarification loops, misroutes, queue age, and escalation use. The record also carries the choice supported by that proof: Redesign routing when review effort is dominated by reconstructing context. Keeping evidence and decision together lets a later owner distinguish a deliberate residual risk from a premise that nobody examined. It also gives the team a clear reason to reopen the choice when the underlying facts change.

Across the work governed by workflow owners, automation leaders, AI program owners, operations executives, and risk teams, occurrences of this pattern should be compared rather than treated as unrelated project stories. The positive standard is: A human handoff needs the case, reason, evidence, attempted actions, affected systems, and deadline. If the same failure recurs, leaders can change delegation, capacity, intake, policy, or the intervention itself. That portfolio response is usually more valuable than asking each team to absorb another local workaround.

Two questions test route with context. What evidence would show that the condition is no longer adequate? Would that evidence arrive early enough to preserve a feasible choice? The intended choice is clear: Redesign routing

when review effort is dominated by reconstructing context. A weak answer to either question calls for a smaller commitment, an earlier signal, or a safer fallback. A strong answer earns movement without claiming that uncertainty has disappeared.

Assign response ownership

Each class needs an owner for the case and an owner for the recurring condition behind it. The visible artifact is often complete before the operating condition behind it is credible.

Operations clear individual cases while no product, policy, data, or process owner addresses the source. The organization may call this execution risk, yet the failure began when authority and evidence were separated at the decision point.

Separate case resolution from systemic remediation and give both owners service expectations and escalation authority. A useful test asks what ordinary operating pressure would break the premise and how that break would be detected before consequence spreads.

Case disposition, remediation backlog, recurrence, breached service levels, and decisions about accepted residual risk. Evidence should arrive before the latest useful decision date; perfect analysis delivered after options expire has little governance value.

Reduce automation scope when systemic exceptions persist without accountable remediation. The operating owner should confirm that the decision can be executed with the people, systems, and time actually available.

A tabletop review of assign response ownership should use the observed break as its scenario: Operations clear individual cases while no product, policy, data, or process owner addresses the source. Participants then apply the proposed response. They separate case resolution from systemic remediation and give both owners service expectations and escalation authority. The review identifies where authority, information, time, or recovery would run out. It is useful only if it changes the boundary or confirms who will act when the condition appears.

The minimum record for assign response ownership is anchored in actual proof. It includes case disposition, remediation backlog, recurrence, breached service levels, and decisions about accepted residual risk. The record also carries the choice supported by that proof: Reduce automation scope when systemic exceptions persist without accountable remediation. Keeping evidence and decision together lets a later owner distinguish a deliberate residual risk from a premise that nobody examined. It also gives the team a clear reason to reopen the choice when the underlying facts change.

Across the work governed by workflow owners, automation leaders, AI program owners, operations executives, and risk teams, occurrences of this pattern should be compared rather than treated as unrelated project stories. The positive standard is: Each class needs an owner for the case and an owner for the recurring condition behind it. If the same failure recurs, leaders can change delegation, capacity, intake, policy, or the intervention itself. That portfolio response is usually more valuable than asking each team to absorb another local workaround.

Two questions test assign response ownership. What evidence would show that the condition is no longer adequate? Would that evidence arrive early enough to preserve a feasible choice? The intended choice is clear: Reduce automation scope when systemic exceptions persist without accountable remediation. A weak answer to either question calls for a smaller commitment, an earlier signal, or a safer fallback. A strong answer earns movement without claiming that uncertainty has disappeared.

Recover to a known state

Resolution must restore data, systems, customer commitments, and workflow state—not simply close a ticket. The issue is not a shortage of activity; it is a shortage of inspectable commitments.

A case is marked resolved after a manual workaround while partial transactions, duplicate records, or inconsistent downstream states remain. The missing condition is often treated as something the team will resolve later, even though the team lacks the authority or capacity to resolve it.

Define rollback, correction, reconciliation, communication, approval, and safe re-entry for each material exception. The forum should be able to distinguish an acceptable residual risk from a missing obligation that has merely been renamed.

Recovery time, reconciliation results, repeat contact, residual inconsistency, and successful exercises before live incidents. The evidence may be compact, but it must be inspectable. A future reviewer should be able to see what was examined, what conclusion was drawn, and which limitation remained.

Keep actions reversible until recovery is demonstrated under representative conditions. Leaders should prefer the smallest commitment that preserves learning and limits irreversible exposure.

A tabletop review of recover to a known state should use the observed break as its scenario: A case is marked resolved after a manual workaround while partial transactions, duplicate records, or inconsistent downstream states remain. Participants then apply the proposed response. They define rollback, correction, reconciliation, communication, approval, and safe re-entry for each material exception. The review identifies where authority, information, time, or recovery would run out. It is useful only if it changes the boundary or confirms who will act when the condition appears.

The minimum record for recover to a known state is anchored in actual proof. It includes recovery time, reconciliation results, repeat contact, residual inconsistency, and successful exercises before live incidents. The record also carries the choice supported by that proof: Keep actions reversible until recovery is demonstrated under representative conditions. Keeping evidence and decision together lets a later owner distinguish a deliberate residual risk from a premise that nobody examined. It also gives the team a clear reason to reopen the choice when the underlying facts change.

Across the work governed by workflow owners, automation leaders, AI program owners, operations executives, and risk teams, occurrences of this pattern should be compared rather than treated as unrelated project stories. The positive standard is: Resolution must restore data, systems, customer commitments, and workflow state—not simply close a ticket. If the same failure recurs, leaders can change delegation, capacity, intake, policy, or the intervention itself. That portfolio response is usually more valuable than asking each team to absorb another local workaround.

Two questions test recover to a known state. What evidence would show that the condition is no longer adequate? Would that evidence arrive early enough to preserve a feasible choice? The intended choice is clear: Keep actions reversible until recovery is demonstrated under representative conditions. A weak answer to either question calls for a smaller commitment, an earlier signal, or a safer fallback. A strong answer earns movement without claiming that uncertainty has disappeared.

Learn and redesign

Exception evidence should change rules, data, interfaces, training, permissions, and the definition of eligible work. The cost appears downstream, but the missing decision sits upstream.

Teams celebrate declining manual review after raising thresholds or hiding failures rather than removing causes. The recurring signal is translation work: people spend time discovering what was meant, who may decide, and how to repair the unsupported assumption.

Review volume, severity, recurrence, escape, and specialist effort; sample closed cases and add important ones to evaluation. This does not require a larger pack. It requires a traceable connection among the premise, the evidence, the boundary, and the action that follows.

Cause distribution, cases prevented, control changes, new test cases, downstream outcomes, and unresolved novel classes. Evidence has a shelf life. The record needs the conditions or date that would make the conclusion stale.

Expand the happy path only when the exception path shows the operation can carry the additional reliance. A hold is governable when the return conditions are explicit; otherwise it becomes an unmanaged queue.

A tabletop review of learn and redesign should use the observed break as its scenario: Teams celebrate declining manual review after raising thresholds or hiding failures rather than removing causes. Participants then apply the proposed response. They review volume, severity, recurrence, escape, and specialist effort; sample closed cases and add important ones to evaluation. The review identifies where authority, information, time, or recovery would run out. It is useful only if it changes the boundary or confirms who will act when the condition appears.

The minimum record for learn and redesign is anchored in actual proof. It includes cause distribution, cases prevented, control changes, new test cases, downstream outcomes, and unresolved novel classes. The record also carries the choice supported by that proof: Expand the happy path only when the exception path shows the operation can carry the additional reliance. Keeping evidence and decision together lets a later owner distinguish a deliberate residual risk from a premise that nobody examined. It also gives the team a clear reason to reopen the choice when the underlying facts change.

Across the work governed by workflow owners, automation leaders, AI program owners, operations executives, and risk teams, occurrences of this pattern should be compared rather than treated as unrelated project stories. The positive standard is: Exception evidence should change rules, data, interfaces, training, permissions, and the definition of eligible work. If the same failure recurs, leaders can change delegation, capacity, intake, policy, or the intervention itself. That portfolio response is usually more valuable than asking each team to absorb another local workaround.

Two questions test learn and redesign. What evidence would show that the condition is no longer adequate? Would that evidence arrive early enough to preserve a feasible choice? The intended choice is clear: Expand the happy path only when the exception path shows the operation can carry the additional reliance. A weak answer to either question calls for a smaller commitment, an earlier signal, or a safer fallback. A strong answer earns movement without claiming that uncertainty has disappeared.

How the elements interact

Classify exceptions and Detect and contain protect different parts of the same commitment. An exception taxonomy should describe why ordinary processing is unsafe or impossible and what kind of response is required. The system must recognize boundary conditions early and move into a safe state. The visible artifact is often complete before the operating condition behind it is credible. If the operation encounters this failure—every failure enters one generic manual-review queue, hiding severity, recurrence, and the skills needed to resolve it.—the response for detect and contain cannot compensate for the missing classify exceptions obligation. Reviewers should reconcile the evidence for both elements before they accept the next action: stop the case—and sometimes the workflow—when detection or containment is unreliable.

Detect and contain and Route with context protect different parts of the same commitment. The system must recognize boundary conditions early and move into a safe state. A human handoff needs the case, reason, evidence, attempted actions, affected systems, and deadline. A useful governance design therefore begins with the consequence leaders are prepared to accept. If the operation encounters this failure—it continues with a plausible guess, partially completes an action, or fails silently because confidence and policy checks are not operationalized.—the response for route with context cannot compensate for the missing detect and contain obligation. Reviewers should reconcile the evidence for both elements before they accept the next action: redesign routing when review effort is dominated by reconstructing context.

Route with context and Assign response ownership protect different parts of the same commitment. A human handoff needs the case, reason, evidence, attempted actions, affected systems, and deadline. Each class needs an owner for the case and an owner for the recurring condition behind it. This is where a management ritual either becomes a real control or reveals itself as administrative theater. If the operation encounters this failure—reviewers receive a vague alert and reconstruct the history across logs, messages, and systems before they can act.—the response for assign response ownership cannot compensate for the missing route with context obligation. Reviewers should reconcile the evidence for both elements before they accept the next action: reduce automation scope when systemic exceptions persist without accountable remediation.

Assign response ownership and Recover to a known state protect different parts of the same commitment. Each class needs an owner for the case and an owner for the recurring condition behind it. Resolution must restore data, systems, customer commitments, and workflow state—not simply close a ticket. The issue is not a shortage of activity; it is a shortage of inspectable commitments. If the operation encounters this failure—operations clear individual cases while no product, policy, data, or process owner addresses the source.—the response for recover to a known state cannot compensate for the missing assign response ownership obligation. Reviewers should reconcile the evidence for both elements before they accept the next action: keep actions reversible until recovery is demonstrated under representative conditions.

Recover to a known state and Learn and redesign protect different parts of the same commitment. Resolution must restore data, systems, customer commitments, and workflow state—not simply close a ticket. Exception evidence should change rules, data, interfaces, training, permissions, and the definition of eligible work. A strong operating model makes the hidden negotiation explicit while options are still available. If the operation encounters this failure—a case is marked resolved after a manual workaround while partial transactions, duplicate records, or inconsistent downstream states remain.—the response for learn and redesign cannot compensate for the missing recover to a known state obligation. Reviewers should reconcile the evidence for both elements before they accept the next action: expand the happy path only when the exception path shows the operation can carry the additional reliance.

Learn and redesign and Classify exceptions protect different parts of the same commitment. Exception evidence should change rules, data, interfaces, training, permissions, and the definition of eligible work. An exception taxonomy should describe why ordinary processing is unsafe or impossible and what kind of response is required. The work becomes governable when ownership, evidence, boundaries, and response are connected to a real choice. If the operation encounters this failure—teams celebrate declining manual review after raising thresholds or hiding failures rather than removing causes.—the response for classify exceptions cannot compensate for the missing learn and redesign obligation. Reviewers should reconcile the evidence for both elements before they accept the next action: separate routine recoverable cases from events requiring immediate containment or policy decision.

Put the model into the management cadence

A framework becomes useful when it changes the normal cadence of work. It should shape intake, funding, design, delivery, and operating review without creating a separate governance industry around itself. The following sequence is deliberately small:

EXHIBIT 2

Start with one decision, then calibrate from evidence

1. **Move 1**
   Build the taxonomy from real historical cases and frontline interviews.
2. **Move 2**
   Design detection, containment, routing, ownership, evidence, and recovery for each high-consequence class.
3. **Move 3**
   Exercise the path with missing, ambiguous, conflicting, and unsafe inputs before launch.
4. **Move 4**
   Review exception demand as a capacity, product, and operations signal.

- Build the taxonomy from real historical cases and frontline interviews.
- Design detection, containment, routing, ownership, evidence, and recovery for each high-consequence class.
- Exercise the path with missing, ambiguous, conflicting, and unsafe inputs before launch.
- Review exception demand as a capacity, product, and operations signal.

The first cycle should be treated as calibration. Reviewers should note which questions changed a decision, which evidence was unavailable, where authority was unclear, and which controls cost out of proportion to the exposure. That record improves the six-part exception path without pretending the first version will fit every workflow or investment.

Ownership should remain close to the consequence. The PMO or AI-governance function can maintain the method, surface cross-portfolio patterns, and challenge weak evidence. It should not absorb the business owner's accountability for the result or the executive's accountability for the commitment.

Measures that reveal whether the control works

- Time from a material signal to a consequential decision.
- Commitments narrowed, redirected, paused, or stopped before avoidable exposure grew.
- Exceptions or assumptions discovered after the latest useful decision date.
- Scarce specialist and executive attention consumed by recurring unresolved conditions.
- Decisions reopened because the original evidence, boundary, or rationale was unclear.
- Operating outcomes and recovery performance after reliance expanded.

These measures are diagnostic, not a universal scorecard. Their purpose is to identify where the operating model fails repeatedly and whether governance is preserving options earlier. A lower count can indicate improvement, but it can also indicate weak detection. Leaders should read the measures with case evidence and frontline observation.

The strongest counterargument

Not every rare event deserves bespoke automation. Some exceptions should remain human-owned, and some cases should be excluded entirely. The objective is not zero exceptions; it is a deliberate path that contains consequence and makes learning possible.

A reasonable critic could also argue that experienced leaders already do this through judgment. Many do. The weakness is not the absence of judgment; it is the absence of a durable operating record when people, pressure, and conditions change. The six-part exception path makes the basis for judgment visible enough to challenge, execute, and revisit.

The shift

The happy path demonstrates throughput. The exception path demonstrates whether the operation can detect what falls outside the rules, route it to an accountable owner, bound the consequence, preserve evidence, recover, and learn. The practical shift is from reporting activity to governing the conditions under which the organization relies on that activity.

That discipline does not make uncertainty disappear. It gives leaders a better place to stand: a named decision, evidence proportionate to consequence, an explicit boundary, a responsible owner, and a response when reality departs from the assumption. Those elements are enough to turn hidden operating risk into a choice the portfolio can make deliberately.

Notes and sources

[1] NIST, "AI Risk Management Framework Core," 2023. https://airc.nist.gov/airmf-resources/airmf/5-sec-core/

[2] Blake Bullwinkel et al., "Lessons From Red Teaming 100 Generative AI Products," Microsoft Research, January 2025. https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani